

lezione-14-02-24

September 1, 2024

####

WebScraping

Anno Accademico 2022-2023

Docente: Laura Ricci

Lezione 5

Pandas: Series e DataFrames

14 Febbraio 2024

0.1 Pandas

- libreria per la **data analysis** costruita su *Python* e altre librerie
- utilizza routines C per la realizzazione efficace di molte funzionalità
- open source
- compatibile con **Excel** e **Google's sheets**
- come in questi framework, *Pandas* permette di interagire con dati rappresentati come *tabelle*, formate da righe e da colonne
- offre diverse funzionalità per lavorare su queste strutture
 - indexing
 - sorting,
 - filtering
 - cleaning
 - aggregating
 - pivoting
 - joining
 - deduping

0.2 Pandas

- offre anche la possibilità di *caricare* e *salvare* dati in formati tabulari “standard”
 - **CSV** (Comma-separated Values)
 - **TSV** (Tab-separated values)
 - File **Excel**
 - Formati database *...
- operazioni numeriche e statistiche efficienti
- primitive di visualizzazione applicabili direttamente a strutture *Pandas*

0.3 Pandas: la gerarchia dei moduli

0.4 Pandas: Tipi di dato

- due tipi di dati principali
 - le **Series** rappresentano dati *1D*
 - * ottime per rappresentare sequenze temporali
 - i **Data Frame** rappresentano dati *2D* come un file csv, una tabella di un database, ...
- quale è la relazione tra i due?
 - ogni colonna di un *DataFrame* è una *Series*
- lavoreremo con entrambe le strutture esaminando data-set reali

0.5 Le Series

- array monodimensionale
 - una sequenza *ordinata* di valori
 - elementi accessibili per **label** o per **posizione**
 - dati omogenei
- *Pandas* assegna ad ogni valore contenuto in una **Series**
 - una label, ovvero un **label index** utilizzato per localizzare un elemento
 - la **posizione** dell'elemento nella serie (il primo elemento ha indice 0), ovvero un **position index**
- quale è l'idea di base?
 - combinare caratteristiche di **liste** e di **dizionari**
 - come in una lista, gli elementi sono accessibili per indice
 - come in un dizionario si può accedere ad un elemento usando la label

0.6 Le Series

```
[1]: import numpy as np
import pandas as pd
```

C:\Users\ricci\AppData\Local\Temp\ipykernel_12300\3269992828.py:2:

DeprecationWarning:

Pyarrow will become a required dependency of pandas in the next major release of pandas (pandas 3.0),

(to allow more performant data types, such as the Arrow string type, and better interoperability with other libraries)

but was not found to be installed on your system.

If this would cause problems for you,

please provide us feedback at <https://github.com/pandas-dev/pandas/issues/54466>

```
import pandas as pd
```

```
[2]: pd.Series()
```

```
[2]: Series([], dtype: object)
```

- **pd.Series** è una classe della libreria Pandas

- invocando `pd.Series()` invoco il costruttore e creo un oggetto di quella classe, vuoto
- poichè non ho inserito elementi, Pandas per default assegna un tipo degli elementi uguale a `float64`

0.7 Le Series: creazione a partire da oggetti Python

```
[3]: # importing pandas as pd
import pandas as pd

ice_cream_flavors = ["Chocolate", "Vanilla", "Strawbrrry", "Rum Raisin",]

# Creazione di una serie a partire da una lista Python

ics=pd.Series(ice_cream_flavors)

print(ics)
```

```
0    Chocolate
1     Vanilla
2   Strawbrrry
3    Rum Raisin
dtype: object
```

- Il primo modo di inizializzare una series è partire da un oggetto *Python*
- creazione di una series a partire da una lista
 - notare che l'ordine degli elementi della lista è conservato
- il primo dei parametri del costruttore della classe Series è la struttura contenente i dati da inserire nella serie
- ma ci sono altri parametri? Quali parametri consentono di configurare le caratteristiche della *Series*?
- c'è un modo semplice e soprattutto interattivo per vedere quali sono i parametri, direttamente dal Notebook?

0.8 Le Series: parametri del costruttore

- per ottenere informazioni sui parametri del costruttore: premere **Shift + TAB**
- viene visualizzato ogni parametro insieme ad un valore **di default**, utilizzato se non si fornisce alcun valore del parametro attuale
- ogni parametro che ha un valore di default, è, per definizione, **opzionale**

0.9 Le Series: posizione e indice

- ogni elemento della Series è identificato da una posizione (positional index) e da una label (label index)
- l'insieme delle label definisce il **label index** della serie, spesso indicato come **indice**
- label e posizione di ogni elemento coincidono, se non si definisce esplicitamente un valore per le label

```
[4]: ics.index
```

```
[4]: RangeIndex(start=0, stop=4, step=1)
```

- è possibile settare esplicitamente un **label index** della Series, come secondo parametro del costruttore
- deve essere un tipo immutabile (string, tupla, datetimes,...)

```
[5]: days_of_week = ['Monday', 'Wednesday', 'Friday', 'Saturday']

apd.Series(ice_cream_flavors, days_of_week)
```

```
[5]: Monday      Chocolate
     Wednesday   Vanilla
     Friday      Strawbrry
     Saturday    Rum Raisin
     dtype: object
```

0.10 Series: “under the wood”

- *Pandas*
 - delega la memorizzazione dei *valori* contenuti in una Series a *NumPy*
 - * utilizza *ndarray*: struttura dati di *NumPy* usata per rappresentare matrici (a sua volta utilizza routine C)
 - * scopo: utilizzare le operazioni vettoriali di *NumPy*, molto efficienti
 - * *Series* come wrapper di oggetti *NumPy*
 - invece, gli attributi degli indici sono memorizzati in un oggetto *Index* di *Pandas*
- una *Series* è quindi un oggetto che ne contiene altri due

0.11 Python: attributi degli oggetti

- gli oggetti *Python* sono caratterizzati da una serie di *attributi*, di *classe* e di *istanza*
- gli attributi sono acceduti con la notazione puntata
- attributi di classe: condivisi da tutte le istanze di quella classe

```
[6]: class Car:
     class_wheels = 4 # 'wheels' is a class attribute

car1 = Car()
car2 = Car()

print(car1.class_wheels)
print(car2.class_wheels)
```

4

4

0.12 Python: attributi degli oggetti

- esempio di attributo di istanza

```
[7]: class Car:
      def __init__(self, color):
          self.color = color # 'color' is an instance attribute

      car1 = Car("red")
      car2 = Car("blue")

      print(car1.color)
      print(car2.color)
```

```
red
blue
```

0.13 Series: “under the wood”

- gli attributi possono essere utilizzati per analizzare come è strutturato un oggetto di *Pandas*
- analizziamo gli attributi della nostra serie *ics*
- verifichiamo che la *series* memorizza i suoi valori in un array *NumPy*

```
[8]: print(ics.values)
      type(ics.values)
```

```
['Chocolate' 'Vanilla' 'Strawbwrry' 'Rum Raisin']
```

```
[8]: numpy.ndarray
```

```
[9]: print(ics.index)
      type(ics.index)
```

```
RangeIndex(start=0, stop=4, step=1)
```

```
[9]: pandas.core.indexes.range.RangeIndex
```

```
[10]: info_calorie ={
      "Cerali":125,
      "Chocolate" : 406,
      "IceCream": 342
      }
      dieta = pd.Series(info_calorie)
      dieta
```

```
[10]: Cerali      125
      Chocolate  406
      IceCream   342
      dtype: int64
```

0.14 Series: creazione a partire da oggetti Python

- una serie può essere creata a partire da oggetti *Python*, in maniera diretta o indiretta.
- creazione a partire da:
 - **dizionari**:
 - per ogni chiave del dizionario, una corrispondente label della serie
 - **tuple**
 - indice posizionale e label coincidono
 - **array NumPy**
- un **set** non definisce un ordinamento: occorre convertirlo in una struttura dati che definisce un ordinamento

0.15 Series: data selection dictionary-like

```
[11]: import pandas as pd
population_dict = {'India': 1366417754,
                  'India': 1466428893,
                  'China': 1397715000,
                  'USA': 328239523,
                  'England': 55977200,
                  'Russia': 143666931,
                  'Japan': 126264931}
population = pd.Series(population_dict)
print(population)
```

```
India      1466428893
China      1397715000
USA         328239523
England     55977200
Russia     143666931
Japan      126264931
dtype: int64
```

```
[12]: population['USA']
```

```
[12]: 328239523
```

```
[13]: population.keys()
```

```
[13]: Index(['India', 'China', 'USA', 'England', 'Russia', 'Japan'], dtype='object')
```

```
[14]: list(population.items())
```

```
[14]: [('India', 1466428893),
      ('China', 1397715000),
      ('USA', 328239523),
      ('England', 55977200),
      ('Russia', 143666931),
```

```
('Japan', 126264931)]
```

0.16 Series: data selection one dimensional array-like

- come gli array: *accesso diretto, slicing, masking*

```
[15]: #slicing by label index  
population['India':'USA']
```

```
[15]: India    1466428893  
      China    1397715000  
      USA      328239523  
      dtype: int64
```

```
[16]: #slicing by positional index  
population[0:2]
```

```
[16]: India    1466428893  
      China    1397715000  
      dtype: int64
```

```
[17]: # masking  
population[(population>147000000)]
```

```
[17]: India    1466428893  
      China    1397715000  
      USA      328239523  
      dtype: int64
```

```
[18]: population [['China', 'Russia']]
```

```
[18]: China      1397715000  
      Russia    143666931  
      dtype: int64
```

0.17 Series: indexers, loc e iloc

- introdotti per evitare confusione nel caso si usino label numeriche
- in questo caso si potrebbe generare confusione tra *label index* e *positional index*

```
[19]: Regions = pd.Series(['Toscana', 'Sicilia', 'Puglia', 'Sardegna'],  
                          index=[2,4,6,8] )
```

- cosa accade se accedo mediante indice numerico? Considero posizione o label?

```
[20]: Regions = pd.Series(['Toscana', 'Sicilia', 'Puglia', 'Sardegna'],  
                          index=[2,4,6,8] )
```

```
print(Regions[2])
```

Toscana

- le operazioni di accesso utilizzano la label, le operazioni di slicing gli indici numerici

```
[21]: Regions[1:3]
```

```
[21]: 4    Sicilia
      6    Puglia
      dtype: object
```

0.18 Series: indexers, loc e iloc

- per evitare ambiguità Pandas introduce due *indexers attribute* che indicano esplicitamente quale schema si vuole utilizzare
- accesso per label: *loc*
- utilizzato per accesso diretto o **slicing**

```
[22]: Regions.loc[2]
```

```
[22]: 'Toscana'
```

```
[23]: Regions.loc[1:4]
```

```
[23]: 2    Toscana
      4    Sicilia
      dtype: object
```

- accesso per posizione: *iloc*

```
[24]: Regions.iloc[1]
```

```
[24]: 'Sicilia'
```

```
[25]: Regions.iloc[1:4]
```

```
[25]: 4    Sicilia
      6    Puglia
      8    Sardegna
      dtype: object
```

0.19 Series: statistica descrittiva

```
[26]: numbers = pd.Series([1,2,3,np.nan,4,5])
      numbers
```



```
[26]: 0    1.0
      1    2.0
      2    3.0
      3   NaN
      4    4.0
      5    5.0
      dtype: float64
```

```
[27]: numbers.cumsum()
```

```
[27]: 0    1.0
      1    3.0
      2    6.0
      3   NaN
      4   10.0
      5   15.0
      dtype: float64
```

```
[28]: numbers.cumsum(skipna=False)
```

```
[28]: 0    1.0
      1    3.0
      2    6.0
      3   NaN
      4   NaN
      5   NaN
      dtype: float64
```

0.20 Series: operazioni aritmetiche, broadcasting

- le operazioni aritmetiche possono utilizzare la tecnica del **broadcasting** definita da *NumPy*
- consente di definire operazioni su intere serie, applicando una operazione elementare ad ogni elemento della *Series*

```
[29]: s1 = pd.Series([1,2,3],index=["A","B","C"])
      s2 = pd.Series([4,5,6], index=["A","B","C"])
      s1+s2
```

```
[29]: A    5
      B    7
      C    9
      dtype: int64
```

0.21 Series: operazioni aritmetiche, broadcasting

```
[30]: s1 = pd.Series(  
        data = [5, 10, 15], index = ["A", "B", "C"]  
    )  
s2 = pd.Series(  
        data = [4, 8, 12, 14], index = ["B", "C", "D", "E"]  
    )  
s1+s2
```

```
[30]: A      NaN  
      B      14.0  
      C      23.0  
      D      NaN  
      E      NaN  
      dtype: float64
```

0.22 Importare un data set in una Series da un file CSV

- *csv*: un formato semplice, testuale, utilizzato per scambiare dati
- plain-text usato per rappresentare dati tabulari
- la prima riga contiene i nomi delle colonne della tabella
- ogni riga contiene un numero finito di elementi che corrispondono alle diverse colonne
- ogni elemento separato dal successivo da una virgola
- può essere creato a partire da un file *Excel*

0.23 Un DataSet di temperature minime

- descrive le temperature minime della città di Melbourne (Australia)
- considerati 10 anni (1981-1990)
- rilevazioni giornaliere
- sorgente dei dati: *Australian Bureau of Meteorology*
- file: **daily-min-temperatures.csv**

0.24 Importare un DataSet: la funzione read_csv

```
[35]: from pandas import read_csv  
      from matplotlib import pyplot  
      DF = read_csv('DataSets/daily-min-temperatures.csv', parse_dates=True)  
      print(type(DF))  
      DF
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
[35]:      Date  Temp  
0    1981-01-01  20.7  
1    1981-01-02  17.9
```

```

2      1981-01-03  18.8
3      1981-01-04  14.6
4      1981-01-05  15.8
...      ...      ...
3645   1990-12-27  14.0
3646   1990-12-28  13.6
3647   1990-12-29  13.5
3648   1990-12-30  15.7
3649   1990-12-31  13.0

```

```
[3650 rows x 2 columns]
```

- per default, *Pandas* ricerca il dataset nella stessa directory del Notebook
- il DataSet è importato, per default, in una struttura dati a due dimensioni, il **DataFrame**, che vedremo successivamente

0.25 Importare un DataSet in una Series: trasformare le date

```
read_csv('daily-min-temperatures.csv', parse_dates=True)
```

- il parametro *parse_dates* converte tutte gli elementi della colonna Date del file in un oggetto *datetime*

```
[36]: StringSeries = pd.Series(['10/25/2005', '10/29/2002', '01/01/2001'])
      print(StringSeries)
```

```

0      10/25/2005
1      10/29/2002
2      01/01/2001
dtype: object

```

```
[37]: DateSeries=pd.to_datetime(StringSeries)
      print(DateSeries)
```

```

0      2005-10-25
1      2002-10-29
2      2001-01-01
dtype: datetime64[ns]

```

0.26 Lavorare con le date

- *Pandas* contiene la libreria *datetime* che consente di manipolare oggetti di tipo *data*
- diversi usi
 - time series analysis
 - data filtering in base a specifici intervalli di tempo
 - calcolare differenza tra due date
 - raggruppare in base a mese, giorno,...
- lo utilizzeremo spesso nel corso
- **strftime**: un esempio di funzione utile

```
[38]: from datetime import datetime

# get current date
d = datetime.now().date()
print(d)
print(type(d))

# get weekday
print(d.strftime('%w'))

print(d.strftime('%A'))
```

```
2024-02-14
<class 'datetime.date'>
3
Wednesday
```

0.27 Importare un DataSet in una Series: la funzione read_csv

- vogliamo trasformare il **DataFrame** in un oggetto **Series**

```
[39]: from pandas import read_csv
from matplotlib import pyplot
series = read_csv('DataSets/daily-min-temperatures.csv', index_col='Date',
                 ↳ parse_dates=True).squeeze('columns')
type(series)
series
```

```
[39]: Date
1981-01-01    20.7
1981-01-02    17.9
1981-01-03    18.8
1981-01-04    14.6
1981-01-05    15.8
...
1990-12-27    14.0
1990-12-28    13.6
1990-12-29    13.5
1990-12-30    15.7
1990-12-31    13.0
Name: Temp, Length: 3650, dtype: float64
```

- **squeeze** : il dataset viene trasformato in una **Series**
- indico la colonna del *DataFrame* che voglio diventi il **label index**

0.28 Prime analisi della Seies

```
[40]: series.nunique()
```

```
[40]: 229
```

- restituisce il numero di valori diversi contenuti nella serie
- se un valore compare più di una volta, il valore è contato una sola volta

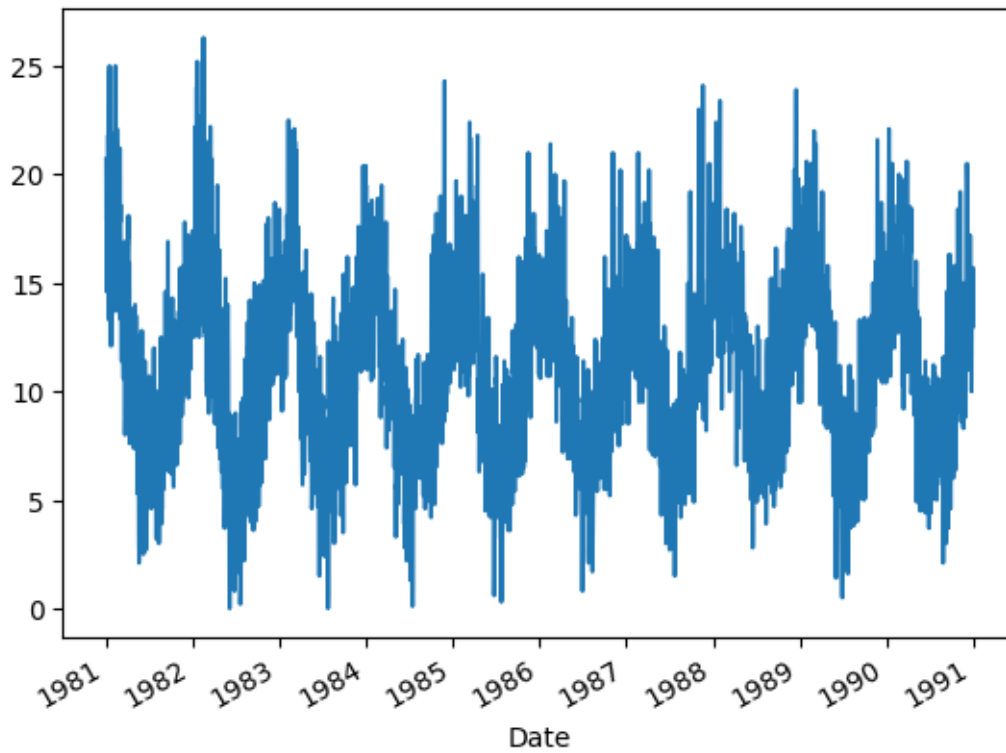
```
[41]: series.value_counts(ascending=True)
```

```
[41]: Temp
21.6      1
21.9      1
0.2       1
0.1       1
24.3      1
      ..
12.5     44
10.5     44
13.0     48
11.0     51
10.0     51
Name: count, Length: 229, dtype: int64
```

- conta il numero di occorrenze di ogni valore della **Series**
- restituisce una **Series** in cui l'**index label** sono i valori delle temperature e i valori sono le rispettive occorrenze nella serie

0.29 Il DataSet delle temperature minime: visualizzazione

```
[42]: series.plot()
      pyplot.show()
```

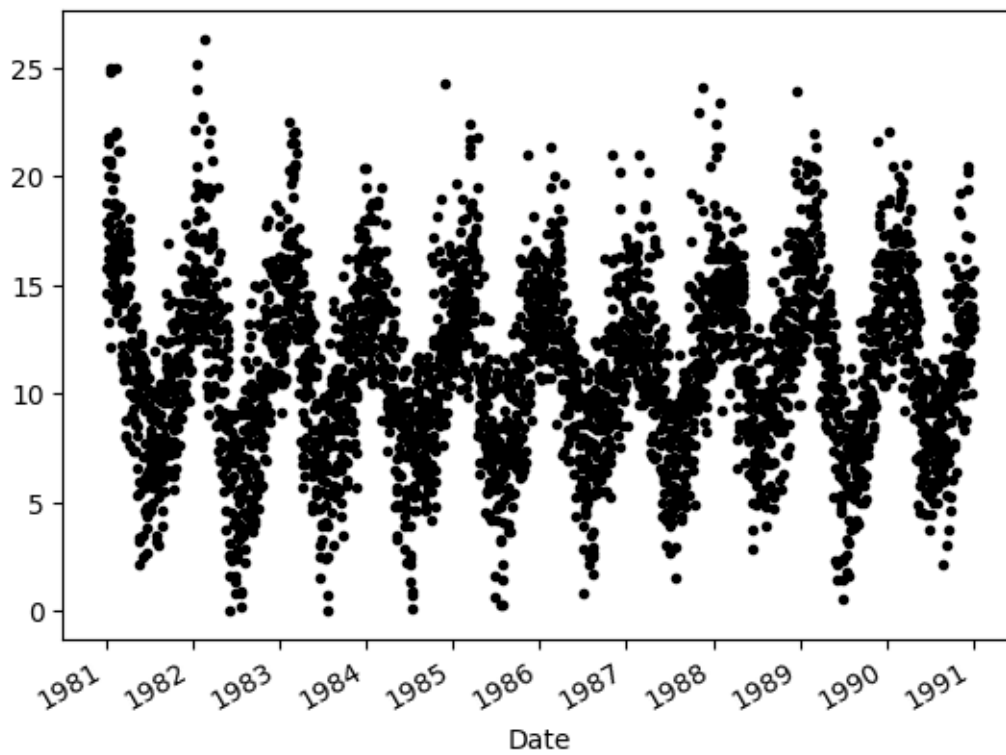


- per default utilizzato un *line plot*

0.30 Il DataSet delle temperature minime: visualizzazione

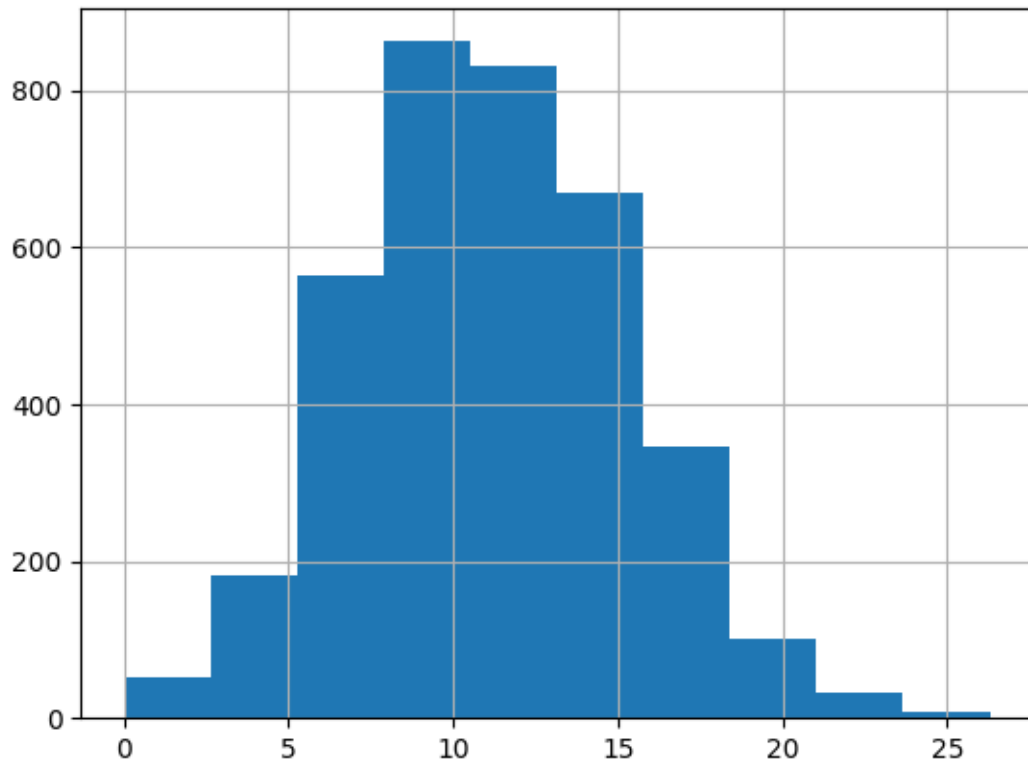
```
[43]: series.plot(style='k.')
```

```
pyplot.show()
```



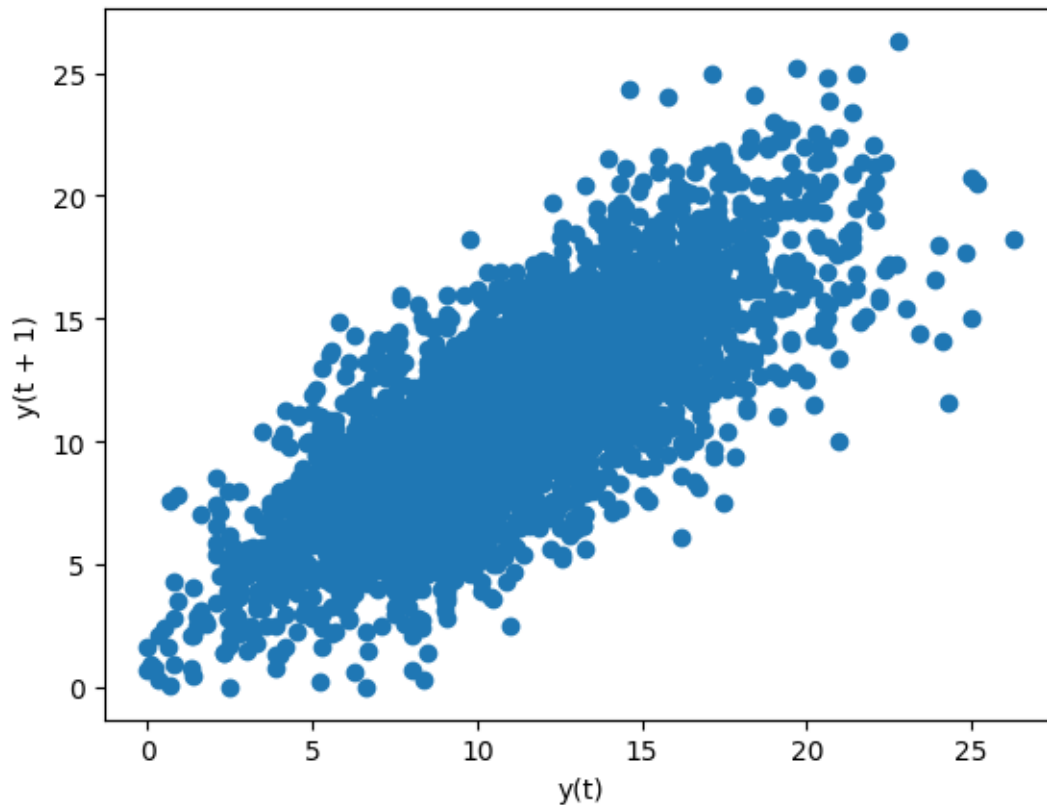
0.31 Il DataSet delle temperature minime: visualizzazione

```
[44]: series.hist()  
      pyplot.show()
```



0.32 Il DataSet delle temperature minime: visualizzazione

```
[45]: from pandas.plotting import lag_plot  
  
lag_plot(series)  
pyplot.show()
```

0.33 Il DataSet delle temperature minime: lag_plot

- visualizza se i valori di una time-series siano random o meno
- dati random non mostrano una struttura identificabile nel lag-plot
- dati non-random mostrano una struttura identificabile

0.34 Il DataSet delle temperature minime: confronto tra anni

```
[46]: from pandas import Grouper
import matplotlib.pyplot as plt

groups = series.groupby(Grouper(freq='YE'))

for name, group in groups:
    print(name)
    print(group)
    print(type(group))
```

1981-12-31 00:00:00

Date

1981-01-01 20.7

```

1981-01-02    17.9
1981-01-03    18.8
1981-01-04    14.6
1981-01-05    15.8
...
1981-12-27    15.5
1981-12-28    13.3
1981-12-29    15.6
1981-12-30    15.2
1981-12-31    17.4
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1982-12-31 00:00:00
Date
1982-01-01    17.0
1982-01-02    15.0
1982-01-03    13.5
1982-01-04    15.2
1982-01-05    13.0
...
1982-12-27    15.3
1982-12-28    16.3
1982-12-29    15.8
1982-12-30    17.7
1982-12-31    16.3
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1983-12-31 00:00:00
Date
1983-01-01    18.4
1983-01-02    15.0
1983-01-03    10.9
1983-01-04    11.4
1983-01-05    14.8
...
1983-12-27    13.9
1983-12-28    11.1
1983-12-29    16.1
1983-12-30    20.4
1983-12-31    18.0
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1984-12-31 00:00:00
Date
1984-01-01    19.5
1984-01-02    17.1
1984-01-03    17.1
1984-01-04    12.0

```

```

1984-01-05    11.0
...
1984-12-26    12.2
1984-12-27    12.0
1984-12-28    12.6
1984-12-29    16.0
1984-12-30    16.4
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1985-12-31 00:00:00
Date
1985-01-01    13.3
1985-01-02    15.2
1985-01-03    13.1
1985-01-04    12.7
1985-01-05    14.6
...
1985-12-27    11.5
1985-12-28    10.8
1985-12-29    12.0
1985-12-30    16.3
1985-12-31    14.4
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1986-12-31 00:00:00
Date
1986-01-01    12.9
1986-01-02    13.8
1986-01-03    10.6
1986-01-04    12.6
1986-01-05    13.7
...
1986-12-27    14.6
1986-12-28    14.2
1986-12-29    13.2
1986-12-30    11.7
1986-12-31    17.2
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1987-12-31 00:00:00
Date
1987-01-01    12.3
1987-01-02    13.8
1987-01-03    15.3
1987-01-04    15.6
1987-01-05    16.2
...
1987-12-27    16.2

```

```

1987-12-28    14.2
1987-12-29    14.3
1987-12-30    13.3
1987-12-31    16.7
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1988-12-31 00:00:00
Date
1988-01-01    15.3
1988-01-02    14.3
1988-01-03    13.5
1988-01-04    15.0
1988-01-05    13.6
...
1988-12-26     9.5
1988-12-27    12.9
1988-12-28    12.9
1988-12-29    14.8
1988-12-30    14.1
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1989-12-31 00:00:00
Date
1989-01-01    14.3
1989-01-02    17.4
1989-01-03    18.5
1989-01-04    16.8
1989-01-05    11.5
...
1989-12-27    13.3
1989-12-28    11.7
1989-12-29    10.4
1989-12-30    14.4
1989-12-31    12.7
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
1990-12-31 00:00:00
Date
1990-01-01    14.8
1990-01-02    13.3
1990-01-03    15.6
1990-01-04    14.5
1990-01-05    14.3
...
1990-12-27    14.0
1990-12-28    13.6
1990-12-29    13.5
1990-12-30    15.7

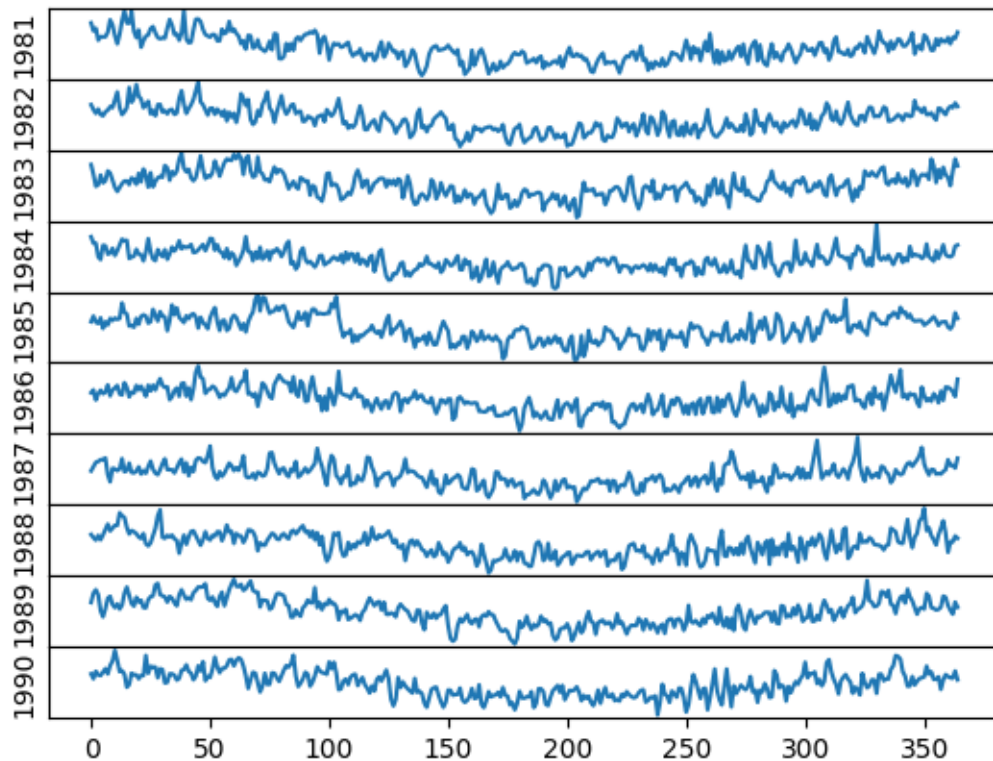
```

```
1990-12-31    13.0
Name: Temp, Length: 365, dtype: float64
<class 'pandas.core.series.Series'>
```

0.35 Il DataSet delle temperature minime: confronto tra anni

```
[47]: fig, axs = plt.subplots(nrows=10,ncols=1,sharex=True)
fig.subplots_adjust(hspace=0)

i=0
for name, group in groups:
    axs[i].plot(group.values)
    axs[i].set_yticks(())
    axs[i].set_ylabel=name.year
    i=i+1
plt.show()
```



0.36 Il DataSet delle temperature minime: heatmap

- visualizzare in una heatmap le rilevazioni di temperatura
- asse x giorni, asse y anni, colore della map più freddo o caldo, a seconda della temperatura
- necessario rappresentare i dati come una matrice **NumPy**

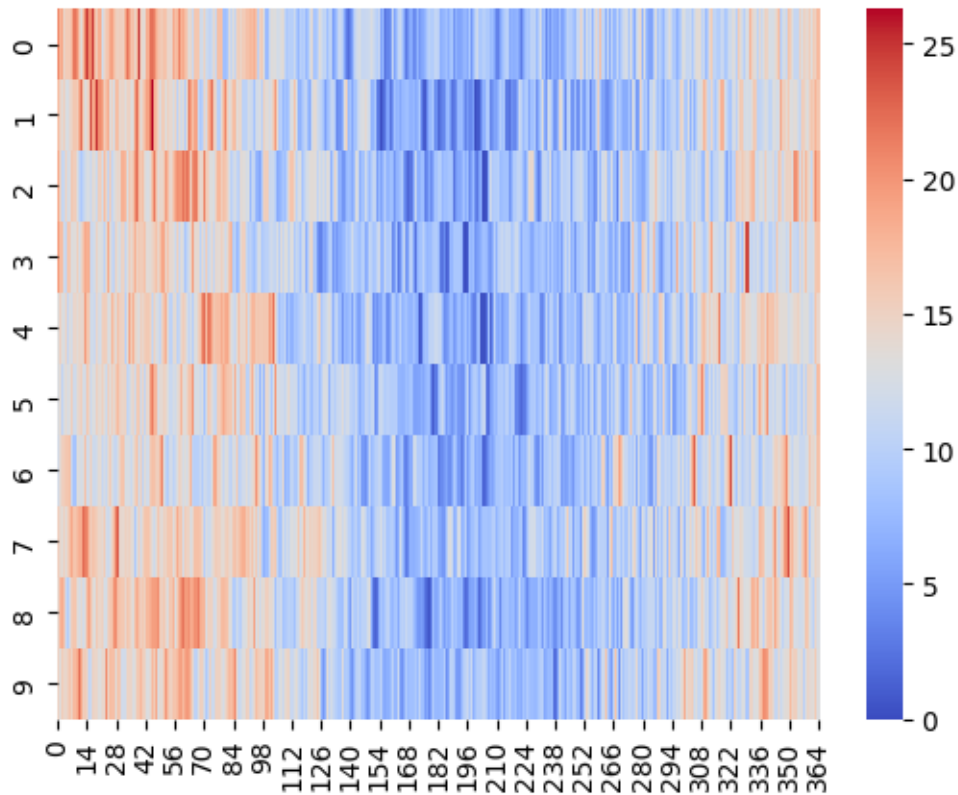
```
[48]: import numpy as np
list=[]
for name, group in groups:
    groupnp=group.to_numpy()
    list.append(groupnp)
matrix=np.asmatrix(list)
print(matrix)
dimensions=np.shape(matrix)
print(dimensions)
```

```
[[20.7 17.9 18.8 ... 15.6 15.2 17.4]
 [17.  15.  13.5 ... 15.8 17.7 16.3]
 [18.4 15.  10.9 ... 16.1 20.4 18. ]
 ...
 [15.3 14.3 13.5 ... 12.9 14.8 14.1]
 [14.3 17.4 18.5 ... 10.4 14.4 12.7]
 [14.8 13.3 15.6 ... 13.5 15.7 13. ]]
(10, 365)
```

0.37 Il DataSet delle temperature minime: heatmap

```
[49]: import seaborn as sns
sns.heatmap(matrix, cmap='coolwarm')
```

```
[49]: <Axes: >
```



- visualizzare in una heatmap le rivazioni di temperatura
- asse x giorni, asse y anni, colore della map più freddo o caldo, a seconda della temperatura

0.38 Il DataSet delle temperature minime: anno 1990, confronto tra i mesi

```
[50]: one_year = series['1990']
one_year_groups = one_year.groupby(Grouper(freq='ME'))

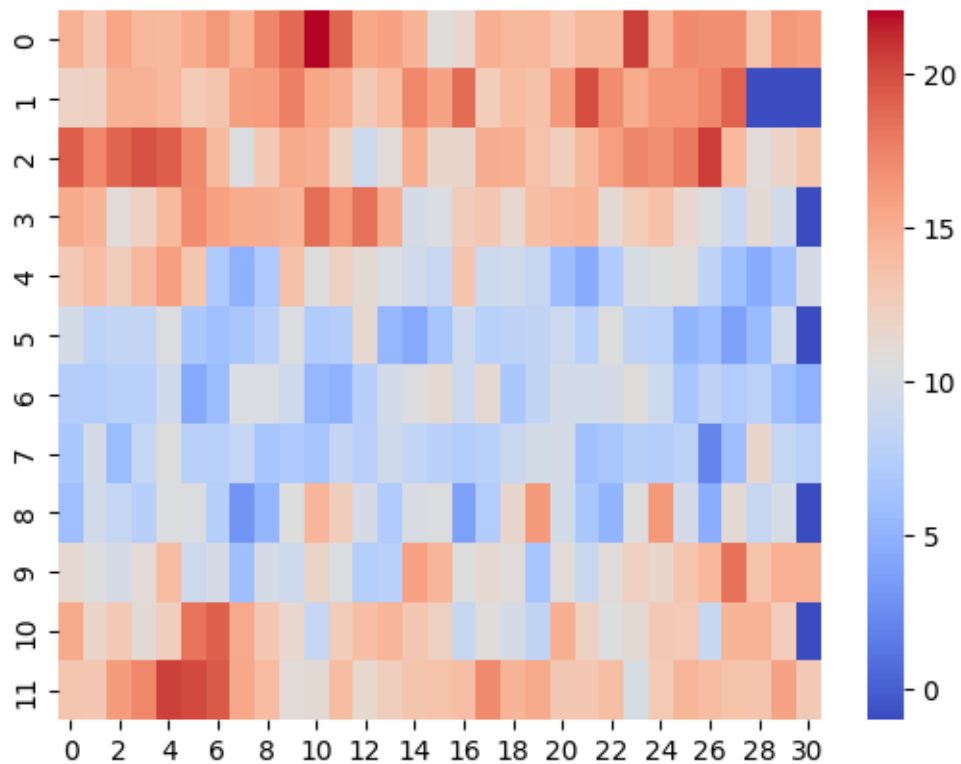
list_month=[]
for name, group in one_year_groups:
    months_np=group.to_numpy()
    for i in range(len(months_np),31):
        months_np= np.append(months_np,-1)
    list_month.append(months_np)

matrix=np.asmatrix(list_month)
dimensions=np.shape(matrix)
print(dimensions)

sns.heatmap(matrix, cmap='coolwarm')
```

(12, 31)

[50]: <Axes: >



0.39 Applicare una funzione ad ogni valore in una Series

- funzioni sono *first class objects* in Python
- possono essere passate come argomento di un'altra funzione
- le *Series* definiscono un metodo **apply**
 - riceve in input il nome di una funzione
 - applica il nome di quella funzione a tutti i valori della serie

```
[51]: series.apply(func = round)
```

```
[51]: Date
1981-01-01    21
1981-01-02    18
1981-01-03    19
1981-01-04    15
1981-01-05    16
...
1990-12-27    14
1990-12-28    14
```



```

1990-12-29    14
1990-12-30    16
1990-12-31    13
Name: Temp, Length: 3650, dtype: int64

```

- ha modificato tutti gli elementi della serie approssimando all'intero superiore

0.40 Pandas: DataFrame

- tabella bidimensionale di dati, con indice di riga e di colonna, simile a uno spreadsheet
- come per le *Series*, ad ogni elemento viene associato un *positionalindex* e un *labelindex*
- le colonne sono oggetti di tipo Series, con le convenzioni di indicizzazione delle Series
- column index (*df.columns*) generalmente è una lista di stringhe o, meno frequentemente, di numeri interi
- row index:
 - interi
 - stringhe
 - valori legati al tempo per time series

0.41 Pandas: data frames

- tre attributi principali
 - **index**: contiene le etichette delle righe
 - **column** contiene le etichette delle colonne
 - **shape** contiene le dimensioni della tabella
- può essere creata a partire da
 - strutture dati Python (un dizionario di **Series**, un dizionario di **list**, una lista di dizionari)
 - leggendo i dati da un file: caso più frequente quando si vuole caricare un data set in un Data Frame
 - * **CSV**
 - * **Excel**
 - * **database MySQL**
 - *(altri formati elencati più avanti)

0.42 Pandas: creare un DataFrame da un dizionario di liste

- ogni elemento *D* del dizionario diventa una colonna del **DataFrame**
 - i nomi delle colonne sono presi dalle chiavi del dizionario
 - valori nella colonna: lista di valori corrispondenti a ogni chiave
 - l'indice di riga è settato, per default, al **position index**
 - l'indice delle varie Series diventa l'indice del Data Frame

```

[ ]: import pandas as pd
df = pd.DataFrame({ "Flower":  ["Violet", "Daisy", "Daffodils",
↪ "Chrysanthemum"],
                    "Month":   ["February", "April", "March", "November"]
                    })

```

```
print(f"{df}\n")
print(f"{df.shape}\n")
print(f"{df.columns}\n")
print(f"{df.index}\n")
```

0.43 Pandas: creare un DataFrame da un dizionario di liste

```
[ ]: df = pd.DataFrame({'angles': [0, 3, 4],
                        'degrees': [360, 180, 360]},
                        index=['circle', 'triangle', 'rectangle'])
df
```

- impostazione esplicita di un *LabelIndex* per le righe

0.44 Pandas: creare un DataFrame da un array NumPy

```
[ ]: random_data = np.random.randint(1,101,[3,5])
random_data
```

```
[ ]: pd.DataFrame(data=random_data)
```

0.45 Pandas: creare un DataFrame da un array NumPy

```
[ ]: row_labels = ["Mattina", "Pomeriggio", "Sera"]
column_labels = ["Lunedì", "Martedì", "Mercoledì", "Giovedì", "Venerdì"]
pd.DataFrame(
    data=random_data,
    index=row_labels,
    columns=column_labels)
```

0.46 il DataSet delle temperature: raggruppare i dati, per anno, in un DataFrame

```
[ ]: from pandas import DataFrame
series = read_csv('DataSets/daily-min-temperatures.csv', index_col='Date',
                 parse_dates=True).squeeze('columns')
groups = series.groupby(Grouper(freq='YE'))
years = pd.DataFrame()
for name, group in groups:
    years[name.year] = group.values
years
```

0.47 il DataSet delle temperature: i boxPlot

0.48 il DataSet delle temperature: boxPlot e outliers

0.49 il DataSet delle temperature: raggruppare i dati, per anno, in un DataFrame

```
[ ]: years.boxplot()  
      pyplot.show()
```

0.50 Importare i dati da un file CSV, l'IRIS Data Set

- uno dei DataSet “storici” e più noti
- 50 campioni presi da ognuna delle 3 specie di *Iris*
 - *Iris Setosa*
 - *Iris Virginica*
 - *Iris Versicolor*
- per ogni campione viene misurata lunghezza e ampiezza
 - dei petali
 - dei sepali
- utilizzato come test case per diverse tecniche di machine learning

0.51 Importare i dati da un file CSV, l'Iris Data Set

```
[ ]: import pandas as pd  
      from pandas.plotting import scatter_matrix  
      import matplotlib.pyplot as plt  
      import os  
      print(os.getcwd())  
      print()  
      dataframe = pd.read_csv('DataSets/IrisDataSet.csv')  
      dataframe
```

0.52 Approfondimento: Struttura del Data Frame

0.53 Una prima analisi del Data Set

```
[ ]: dataframe.shape  
      print(len(dataframe))
```

```
[ ]: dataframe.head()
```

```
[ ]: dataframe.tail()
```

0.54 Una prima analisi del Data Set

```
[ ]: print(dataframe.info())
```

- per capire il tipo degli oggetti contenuti nelle diverse colonne del dataset
- tipo *object*
 - dalla documentazione: “The object data type is “a catch-all for columns that Pandas doesn’t recognize as any other specific type.”
 - in pratica utilizzato per valori di tipo stringa

0.55 Una prima analisi del Data Set

```
[ ]: dataframe.describe()
```

- *df-describe()*
 - fornisce un resoconto sintetico sul contenuto della tabella
 - default: analizza solamente le colonne numeriche
 - per visualizzare informazioni anche sulle colonne non numeriche

```
[ ]: dataframe.describe(include=object)
```

0.56 Una prima analisi del Data Set

```
[ ]: print(dataframe["species"].value_counts())
```

0.57 Plotting Series e DataFrames

```
[ ]: dataframe.hist()
plt.show()
```

0.58 Plotting Series e DataFrames

```
[ ]: dataframe.plot.scatter( x="sepal_width", y="sepal_length")
plt.show()
```

0.59 Plotting Series e DataFrames

```
[ ]: dataframe.plot(kind='box', subplots=True, layout=(2,2), sharex=False,
↪sharey=False)
plt.show()
```

- *box plot*: quartili, mediana, outliers (li vedremo più in dettaglio in seguito)

0.60 Plotting Series e DataFrames

```
[ ]: # scatter plot matrix
scatter_matrix(dataframe)
plt.show()
```

0.61 Plotting Series e DataFrames

```
[ ]: import matplotlib.pyplot as plt
import seaborn as sb
sb.pairplot(dataframe, hue="species")
```

0.62 Assignment 2

- Date le seguenti strutture dati

```
[ ]: superheroes = [
    "Batman",
    "Superman",
    "Spider.man",
    "Iron-Man",
    "Captain America",
    "Wonder Woman"
]
strenght_levels = (100, 120, 90, 95, 110, 120)
```

- utilizzare la lista di superheroes per popolare un oggetto di tipo *Series*
- utilizzare la tupla di strenght_levels per popolare un oggetto di tipo *Series*
- creare una *Series* con i *superheroes* come index labels e *strength_level* come valore. Assegnare la *Series* alla variabile *heroes*
- estrarre i primi due elementi e gli ultimi due elementi della *Series heroes*
- determinare il numero di valori unici di *heroes*
- calcolare la forza media, la massima e la minima di *heroes*
- duplicare il livello di forza di ogni *superhero*
- convertire la *Series heroes* in un dizionario *Python*

0.63 Assignment 3

- E' dato il *dataset revolutionary_war.csv* che contiene diverse battaglie combattute durante la Rivoluzione Americana, con la descrizione del nome della battaglia, la data di inizio, lo stato in cui si è combattuta
- uno storico è interessato a conoscere in quale giorno della settimana sono iniziate il maggior numero di Battaglie durante la rivoluzione
- importare il dataset in una *series* trascurando le colonne che si riferiscono al nome della Battaglia e allo stato, e mantenendo solo la colonna che riguarda la data di inizio della battaglia
- lavorare sulla *series* risultante

```
[ ]:
```